



AP[®] Computer Science A

Practice Exam

The questions contained in this AP[®] Computer Science A Practice Exam are written to the content specifications of AP Exams for this subject. Taking this practice exam should provide students with an idea of their general areas of strengths and weaknesses in preparing for the actual AP Exam. Because this AP Computer Science A Practice Exam has never been administered as an operational AP Exam, statistical data are not available for calculating potential raw scores or conversions into AP grades.

This AP Computer Science A Practice Exam is provided by the College Board for AP Exam preparation. Teachers are permitted to download the materials and make copies to use with their students in a classroom setting only. To maintain the security of this exam, teachers should collect all materials after their administration and keep them in a secure location. Teachers may not redistribute the files electronically for any reason.

Appendix A — A Exam Java Quick Reference

Accessible Methods from the Java Library That May Be Included on the Exam

class java.lang.Object

- boolean equals(Object other)
- String toString()

interface java.lang.Comparable*

- int compareTo(Object other) // returns a value < 0 if this is less than other
// returns a value = 0 if this is equal to other
// returns a value > 0 if this is greater than other

class java.lang.Integer implements java.lang.Comparable*

- Integer(int value)
- int intValue()

class java.lang.Double implements java.lang.Comparable*

- Double(double value)
- double doubleValue()

class java.lang.String implements java.lang.Comparable*

- int length()
- String substring(int from, int to) // returns the substring beginning at from
// and ending at to-1
- String substring(int from) // returns substring(from, length())
- int indexOf(String str) // returns the index of the first occurrence of str;
// returns -1 if not found

class java.lang.Math

- static int abs(int x)
- static double abs(double x)
- static double pow(double base, double exponent)
- static double sqrt(double x)
- static double random() // returns a double in the range [0.0, 1.0)

class java.util.ArrayList<E>

- int size()
- boolean add(E obj) // appends obj to end of list; returns true
- void add(int index, E obj) // inserts obj at position index (0 ≤ index ≤ size),
// moving elements at position index and higher
// to the right (adds 1 to their indices) and adjusts size
- E get(int index)
- E set(int index, E obj) // replaces the element at position index with obj
// returns the element formerly at the specified position
- E remove(int index) // removes element from position index, moving elements
// at position index + 1 and higher to the left
// (subtracts 1 from their indices) and adjusts size
// returns the element formerly at the specified position

*The AP Java subset uses the “raw” Comparable interface, not the generic Comparable<T> interface.

Appendix B — Testable API

info.gridworld.grid.Location class (implements Comparable)

```

public Location(int r, int c)
    constructs a location with given row and column coordinates

public int getRow()
    returns the row of this location

public int getCol()
    returns the column of this location

public Location getAdjacentLocation(int direction)
    returns the adjacent location in the direction that is closest to direction

public int getDirectionToward(Location target)
    returns the closest compass direction from this location toward target

public boolean equals(Object other)
    returns true if other is a Location with the same row and column as this location; false otherwise

public int hashCode()
    returns a hash code for this location

public int compareTo(Object other)
    returns a negative integer if this location is less than other, zero if the two locations are equal, or a positive
    integer if this location is greater than other. Locations are ordered in row-major order.
    Precondition: other is a Location object.

public String toString()
    returns a string with the row and column of this location, in the format (row, col)

```

Compass directions:

```

public static final int NORTH = 0;
public static final int EAST = 90;
public static final int SOUTH = 180;
public static final int WEST = 270;
public static final int NORTHEAST = 45;
public static final int SOUTHEAST = 135;
public static final int SOUTHWEST = 225;
public static final int NORTHWEST = 315;

```

Turn angles:

```

public static final int LEFT = -90;
public static final int RIGHT = 90;
public static final int HALF_LEFT = -45;
public static final int HALF_RIGHT = 45;
public static final int FULL_CIRCLE = 360;
public static final int HALF_CIRCLE = 180;
public static final int AHEAD = 0;

```

info.gridworld.grid.Grid<E> interface

`int getNumRows()`
returns the number of rows, or -1 if this grid is unbounded

`int getNumCols()`
returns the number of columns, or -1 if this grid is unbounded

`boolean isValid(Location loc)`
returns true if loc is valid in this grid, false otherwise
Precondition: loc is not null

`E put(Location loc, E obj)`
puts obj at location loc in this grid and returns the object previously at that location (or null if the location was previously unoccupied).
Precondition: (1) loc is valid in this grid (2) obj is not null

`E remove(Location loc)`
removes the object at location loc from this grid and returns the object that was removed (or null if the location is unoccupied)
Precondition: loc is valid in this grid

`E get(Location loc)`
returns the object at location loc (or null if the location is unoccupied)
Precondition: loc is valid in this grid

`ArrayList<Location> getOccupiedLocations()`
returns an array list of all occupied locations in this grid

`ArrayList<Location> getValidAdjacentLocations(Location loc)`
returns an array list of the valid locations adjacent to loc in this grid
Precondition: loc is valid in this grid

`ArrayList<Location> getEmptyAdjacentLocations(Location loc)`
returns an array list of the valid empty locations adjacent to loc in this grid
Precondition: loc is valid in this grid

`ArrayList<Location> getOccupiedAdjacentLocations(Location loc)`
returns an array list of the valid occupied locations adjacent to loc in this grid
Precondition: loc is valid in this grid

`ArrayList<E> getNeighbors(Location loc)`
returns an array list of the objects in the occupied locations adjacent to loc in this grid
Precondition: loc is valid in this grid

info.gridworld.actor.Actor class

```

public Actor()
    constructs a blue actor that is facing north

public Color getColor()
    returns the color of this actor

public void setColor(Color newColor)
    sets the color of this actor to newColor

public int getDirection()
    returns the direction of this actor, an angle between 0 and 359 degrees

public void setDirection(int newDirection)
    sets the direction of this actor to the angle between 0 and 359 degrees that is equivalent to newDirection

public Grid<Actor> getGrid()
    returns the grid of this actor, or null if this actor is not contained in a grid

public Location getLocation()
    returns the location of this actor, or null if this actor is not contained in a grid

public void putSelfInGrid(Grid<Actor> gr, Location loc)
    puts this actor into location loc of grid gr. If there is another actor at loc, it is removed.
    Precondition: (1) This actor is not contained in a grid (2) loc is valid in gr

public void removeSelfFromGrid()
    removes this actor from its grid.
    Precondition: this actor is contained in a grid

public void moveTo(Location newLocation)
    moves this actor to newLocation. If there is another actor at newLocation, it is removed.
    Precondition: (1) This actor is contained in a grid (2) newLocation is valid in the grid of this actor

public void act()
    reverses the direction of this actor. Override this method in subclasses of Actor to define types of actors with
    different behavior

public String toString()
    returns a string with the location, direction, and color of this actor

```

info.gridworld.actor.Rock class (extends Actor)

```
public Rock()  
    constructs a black rock  
  
public Rock(Color rockColor)  
    constructs a rock with color rockColor  
  
public void act()  
    overrides the act method in the Actor class to do nothing
```

info.gridworld.actor.Flower class (extends Actor)

```
public Flower()  
    constructs a pink flower  
  
public Flower(Color initialColor)  
    constructs a flower with color initialColor  
  
public void act()  
    causes the color of this flower to darken
```

Appendix C — Testable Code for APCS A/AB

Bug.java

```

package info.gridworld.actor;

import info.gridworld.grid.Grid;
import info.gridworld.grid.Location;

import java.awt.Color;

/**
 * A Bug is an actor that can move and turn. It drops flowers as it moves.
 * The implementation of this class is testable on the AP CS A and AB Exams.
 */
public class Bug extends Actor
{
    /**
     * Constructs a red bug.
     */
    public Bug()
    {
        setColor(Color.RED);
    }

    /**
     * Constructs a bug of a given color.
     * @param bugColor the color for this bug
     */
    public Bug(Color bugColor)
    {
        setColor(bugColor);
    }

    /**
     * Moves if it can move, turns otherwise.
     */
    public void act()
    {
        if (canMove())
            move();
        else
            turn();
    }

    /**
     * Turns the bug 45 degrees to the right without changing its location.
     */
    public void turn()
    {
        setDirection(getDirection() + Location.HALF_RIGHT);
    }
}

```

```

/**
 * Moves the bug forward, putting a flower into the location it previously occupied.
 */
public void move()
{
    Grid<Actor> gr = getGrid();
    if (gr == null)
        return;
    Location loc = getLocation();
    Location next = loc.getAdjacentLocation(getDirection());
    if (gr.isValid(next))
        moveTo(next);
    else
        removeSelfFromGrid();
    Flower flower = new Flower(getColor());
    flower.putSelfInGrid(gr, loc);
}

/**
 * Tests whether this bug can move forward into a location that is empty or contains a flower.
 * @return true if this bug can move.
 */
public boolean canMove()
{
    Grid<Actor> gr = getGrid();
    if (gr == null)
        return false;
    Location loc = getLocation();
    Location next = loc.getAdjacentLocation(getDirection());
    if (!gr.isValid(next))
        return false;
    Actor neighbor = gr.get(next);
    return (neighbor == null) || (neighbor instanceof Flower);
    // ok to move into empty location or onto flower
    // not ok to move onto any other actor
}
}

```


BoxBug.java

```
import info.gridworld.actor.Bug;

/**
 * A BoxBug traces out a square "box" of a given size.
 * The implementation of this class is testable on the AP CS A and AB Exams.
 */
public class BoxBug extends Bug
{
    private int steps;
    private int sideLength;

    /**
     * Constructs a box bug that traces a square of a given side length
     * @param length the side length
     */
    public BoxBug(int length)
    {
        steps = 0;
        sideLength = length;
    }

    /**
     * Moves to the next location of the square.
     */
    public void act()
    {
        if (steps < sideLength && canMove())
        {
            move();
            steps++;
        }
        else
        {
            turn();
            turn();
            steps = 0;
        }
    }
}
```

Criticr.java

```

package info.gridworld.actor;

import info.gridworld.grid.Location;
import java.util.ArrayList;

/**
 * A Critter is an actor that moves through its world, processing
 * other actors in some way and then moving to a new location.
 * Define your own critters by extending this class and overriding any methods of this class except for act.
 * When you override these methods, be sure to preserve the postconditions.
 * The implementation of this class is testable on the AP CS A and AB Exams.
 */
public class Critter extends Actor
{
    /**
     * A critter acts by getting a list of other actors, processing that list, getting locations to move to,
     * selecting one of them, and moving to the selected location.
     */
    public void act()
    {
        if (getGrid() == null)
            return;
        ArrayList<Actor> actors = getActors();
        processActors(actors);
        ArrayList<Location> moveLocs = getMoveLocations();
        Location loc = selectMoveLocation(moveLocs);
        makeMove(loc);
    }

    /**
     * Gets the actors for processing. Implemented to return the actors that occupy neighboring grid locations.
     * Override this method in subclasses to look elsewhere for actors to process.
     * Postcondition: The state of all actors is unchanged.
     * @return a list of actors that this critter wishes to process.
     */
    public ArrayList<Actor> getActors()
    {
        return getGrid().getNeighbors(getLocation());
    }
}

```

```

/**
 * Processes the elements of actors. New actors may be added to empty locations.
 * Implemented to "eat" (i.e., remove) selected actors that are not rocks or critters.
 * Override this method in subclasses to process actors in a different way.
 * Postcondition: (1) The state of all actors in the grid other than this critter and the
 * elements of actors is unchanged. (2) The location of this critter is unchanged.
 * @param actors the actors to be processed
 */
public void processActors(ArrayList<Actor> actors)
{
    for (Actor a : actors)
    {
        if (!(a instanceof Rock) && !(a instanceof Critter))
            a.removeSelfFromGrid();
    }
}

/**
 * Gets a list of possible locations for the next move. These locations must be valid in the grid of this critter.
 * Implemented to return the empty neighboring locations. Override this method in subclasses to look
 * elsewhere for move locations.
 * Postcondition: The state of all actors is unchanged.
 * @return a list of possible locations for the next move
 */
public ArrayList<Location> getMoveLocations()
{
    return getGrid().getEmptyAdjacentLocations(getLocation());
}

/**
 * Selects the location for the next move. Implemented to randomly pick one of the possible locations,
 * or to return the current location if locs has size 0. Override this method in subclasses that
 * have another mechanism for selecting the next move location.
 * Postcondition: (1) The returned location is an element of locs, this critter's current location, or null.
 * (2) The state of all actors is unchanged.
 * @param locs the possible locations for the next move
 * @return the location that was selected for the next move.
 */
public Location selectMoveLocation(ArrayList<Location> locs)
{
    int n = locs.size();
    if (n == 0)
        return getLocation();
    int r = (int) (Math.random() * n);
    return locs.get(r);
}

```

```

/**
 * Moves this critter to the given location loc, or removes this critter from its grid if loc is null.
 * An actor may be added to the old location. If there is a different actor at location loc, that actor is
 * removed from the grid. Override this method in subclasses that want to carry out other actions
 * (for example, turning this critter or adding an occupant in its previous location).
 * Postcondition: (1) getLocation() == loc.
 * (2) The state of all actors other than those at the old and new locations is unchanged.
 * @param loc the location to move to
 */
public void makeMove(Location loc)
{
    if (loc == null)
        removeSelfFromGrid();
    else
        moveTo(loc);
}

```

ChameleonCriticr.java

```

import info.gridworld.actor.Actor;
import info.gridworld.actor.Criticr;
import info.gridworld.grid.Location;

import java.util.ArrayList;

/**
 * A ChameleonCriticr takes on the color of neighboring actors as it moves through the grid.
 * The implementation of this class is testable on the AP CS A and AB Exams.
 */
public class ChameleonCriticr extends Criticr
{
    /**
     * Randomly selects a neighbor and changes this critter's color to be the same as that neighbor's.
     * If there are no neighbors, no action is taken.
     */
    public void processActors(ArrayList<Actor> actors)
    {
        int n = actors.size();
        if (n == 0)
            return;
        int r = (int) (Math.random() * n);

        Actor other = actors.get(r);
        setColor(other.getColor());
    }

    /**
     * Turns towards the new location as it moves.
     */
    public void makeMove(Location loc)
    {
        setDirection(getLocation().getDirectionToward(loc));
        super.makeMove(loc);
    }
}

```

Quick Reference A/AB

Location Class (implements Comparable)

```
public Location(int r, int c)
public int getRow()
public int getCol()
public Location getAdjacentLocation(int direction)
public int getDirectionToward(Location target)
public boolean equals(Object other)
public int hashCode()
public int compareTo(Object other)
public String toString()
```

NORTH, EAST, SOUTH, WEST, NORTHEAST, SOUTHEAST, NORTHWEST, SOUTHWEST
LEFT, RIGHT, HALF_LEFT, HALF_RIGHT, FULL_CIRCLE, HALF_CIRCLE, AHEAD

Grid<E> Interface

```
int getNumRows()
int getNumCols()
boolean isValid(Location loc)
E put(Location loc, E obj)
E remove(Location loc)
E get(Location loc)
ArrayList<Location> getOccupiedLocations()
ArrayList<Location> getValidAdjacentLocations(Location loc)
ArrayList<Location> getEmptyAdjacentLocations(Location loc)
ArrayList<Location> getOccupiedAdjacentLocations(Location loc)
ArrayList<E> getNeighbors(Location loc)
```

Actor Class

```
public Actor()
public Color getColor()
public void setColor(Color newColor)
public int getDirection()
public void setDirection(int newDirection)
public Grid<Actor> getGrid()
public Location getLocation()
public void putSelfInGrid(Grid<Actor> gr, Location loc)
public void removeSelfFromGrid()
public void moveTo(Location newLocation)
public void act()
public String toString()
```

Rock Class (extends Actor)

```
public Rock()  
public Rock(Color rockColor)  
public void act()
```

Flower Class (extends Actor)

```
public Flower()  
public Flower(Color initialColor)  
public void act()
```

Bug Class (extends Actor)

```
public Bug()  
public Bug(Color bugColor)  
public void act()  
public void turn()  
public void move()  
public boolean canMove()
```

BoxBug Class (extends Bug)

```
public BoxBug(int n)  
public void act()
```

Critter Class (extends Actor)

```
public void act()  
public ArrayList<Actor> getActors()  
public void processActors(ArrayList<Actor> actors)  
public ArrayList<Location> getMoveLocations()  
public Location selectMoveLocation(ArrayList<Location> locs)  
public void makeMove(Location loc)
```

ChameleonCritter Class (extends Critter)

```
public void processActors(ArrayList<Actor> actors)  
public void makeMove(Location loc)
```

Appendix G: Index for Source Code

This appendix provides an index for the Java source code found in Appendix C.

Bug.java

Bug()	C1
Bug (Color bugColor)	C1
act()	C1
turn()	C1
move()	C2
canMove()	C2

BoxBug.java

BoxBug (int length)	C3
act()	C3

Critter.java

act()	C4
getActors()	C4
processActors (ArrayList<Actor> actors)	C5
getMoveLocations()	C5
selectMoveLocation (ArrayList<Location> locs)	C5
makeMove (Location loc)	C6

ChameleonCritter.java

processActors (ArrayList<Actor> actors)	C6
makeMove (Location loc)	C6

COMPUTER SCIENCE A
SECTION I

Time—1 hour and 15 minutes

Number of questions—40

Percent of total grade—50

Directions: Determine the answer to each of the following questions or incomplete statements, using the available space for any necessary scratch work. Then decide which is the best of the choices given and fill in the corresponding box on the student answer sheet. No credit will be given for anything written in the examination booklet. Do not spend too much time on any one problem.

Notes:

- Assume that the classes listed in the Quick Reference found in the Appendix have been imported where appropriate.
- Assume that declarations of variables and methods appear within the context of an enclosing class.
- Assume that method calls that are not prefixed with an object or class name and are not shown within a complete class definition appear within the context of an enclosing class.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null`.

GO ON TO THE NEXT PAGE.

1. Consider the following method.

```
public static int mystery(int[] arr)
{
    int x = 0;

    for (int k = 0; k < arr.length; k = k + 2)
        x = x + arr[k];

    return x;
}
```

Assume that the array `nums` has been declared and initialized as follows.

```
int[] nums = {3, 6, 1, 0, 1, 4, 2};
```

What value will be returned as a result of the call `mystery(nums)` ?

- (A) 5
- (B) 6
- (C) 7
- (D) 10
- (E) 17

GO ON TO THE NEXT PAGE.

Questions 2-3 refer to the following information.

Consider the following partial class declaration.

```
public class SomeClass
{
    private int myA;
    private int myB;
    private int myC;

    // Constructor(s) not shown

    public int getA()
    { return myA; }

    public void setB(int value)
    { myB = value; }
}
```

2. The following declaration appears in another class.

```
SomeClass obj = new SomeClass();
```

Which of the following code segments will compile without error?

- (A) `int x = obj.getA();`
 - (B) `int x;`
`obj.getA(x);`
 - (C) `int x = obj.myA;`
 - (D) `int x = SomeClass.getA();`
 - (E) `int x = getA(obj);`
-

3. Which of the following changes to `SomeClass` will allow other classes to access but not modify the value of `myC` ?

- (A) Make `myC` public.
- (B) Include the method:
`public int getC()`
`{ return myC; }`
- (C) Include the method:
`private int getC()`
`{ return myC; }`
- (D) Include the method:
`public void getC(int x)`
`{ x = myC; }`
- (E) Include the method:
`private void getC(int x)`
`{ x = myC; }`

GO ON TO THE NEXT PAGE.

4. Consider the following code segment.

```
int x = 7;  
int y = 3;  
  
if ((x < 10) && (y < 0))  
    System.out.println("Value is: " + x * y);  
else  
    System.out.println("Value is: " + x / y);
```

What is printed as a result of executing the code segment?

- (A) Value is: 21
- (B) Value is: 2.3333333
- (C) Value is: 2
- (D) Value is: 0
- (E) Value is: 1

GO ON TO THE NEXT PAGE.

5. Consider the following method.

```
public ArrayList<Integer> mystery(int n)
{
    ArrayList<Integer> seq = new ArrayList<Integer>();

    for (int k = 1; k <= n; k++)
        seq.add(new Integer(k * k + 3));

    return seq;
}
```

Which of the following is printed as a result of executing the following statement?

```
System.out.println(mystery(6));
```

- (A) [3, 4, 7, 12, 19, 28]
- (B) [3, 4, 7, 12, 19, 28, 39]
- (C) [4, 7, 12, 19, 28, 39]
- (D) [39, 28, 19, 12, 7, 4]
- (E) [39, 28, 19, 12, 7, 4, 3]

GO ON TO THE NEXT PAGE.

6. Consider the following method that is intended to determine if the `double` values `d1` and `d2` are close enough to be considered equal. For example, given a `tolerance` of `0.001`, the values `54.32271` and `54.32294` would be considered equal.

```
/** @return true if d1 and d2 are within the specified tolerance,
 *     false otherwise
 */
public boolean almostEqual(double d1, double d2, double tolerance)
{
    /* missing code */
}
```

Which of the following should replace `/* missing code */` so that `almostEqual` will work as intended?

- (A) `return (d1 - d2) <= tolerance;`
- (B) `return ((d1 + d2) / 2) <= tolerance;`
- (C) `return (d1 - d2) >= tolerance;`
- (D) `return ((d1 + d2) / 2) >= tolerance;`
- (E) `return Math.abs(d1 - d2) <= tolerance;`

GO ON TO THE NEXT PAGE.

7. Consider the following class declaration.

```
public class Person
{
    private String myName;
    private int myYearOfBirth;

    public Person(String name, int yearOfBirth)
    {
        myName = name;
        myYearOfBirth = yearOfBirth;
    }

    public String getName()
    { return myName; }

    public void setName(String name)
    { myName = name; }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

Assume that the following declaration has been made.

```
Person student = new Person("Thomas", 1995);
```

Which of the following statements is the most appropriate for changing the name of `student` from "Thomas" to "Tom" ?

- (A) `student = new Person("Tom", 1995);`
- (B) `student.myName = "Tom";`
- (C) `student.getName("Tom");`
- (D) `student.setName("Tom");`
- (E) `Person.setName("Tom");`

GO ON TO THE NEXT PAGE.

8. Consider the following class declaration.

```
public class Student
{
    private String myName;
    private int myAge;

    public Student()
    { /* implementation not shown */ }

    public Student(String name, int age)
    { /* implementation not shown */ }

    // No other constructors
}
```

Which of the following declarations will compile without error?

- I. `Student a = new Student();`
- II. `Student b = new Student("Juan", 15);`
- III. `Student c = new Student("Juan", "15");`

- (A) I only
- (B) II only
- (C) I and II only
- (D) I and III only
- (E) I, II, and III

GO ON TO THE NEXT PAGE.

9. Consider the following method that is intended to return the sum of the elements in the array `key`.

```
public static int sumArray(int[] key)
{
    int sum = 0;

    for (int i = 1; i <= key.length; i++)
    {
        /* missing code */
    }

    return sum;
}
```

Which of the following statements should be used to replace */* missing code */* so that `sumArray` will work as intended?

- (A) `sum = key[i];`
- (B) `sum += key[i - 1];`
- (C) `sum += key[i];`
- (D) `sum += sum + key[i - 1];`
- (E) `sum += sum + key[i];`

Questions 10-11 refer to the following information.

Consider the following instance variable and methods. You may assume that `data` has been initialized with `length > 0`. The methods are intended to return the index of an array element equal to `target`, or -1 if no such element exists.

```
private int[] data;

public int seqSearchRec(int target)
{
    return seqSearchRecHelper(target, data.length - 1);
}

private int seqSearchRecHelper(int target, int last)
{
    // Line 1

    if (data[last] == target)
        return last;
    else
        return seqSearchRecHelper(target, last - 1);
}
```

10. For which of the following test cases will the call `seqSearchRec(5)` always result in an error?

- I. `data` contains only one element.
- II. `data` does not contain the value 5.
- III. `data` contains the value 5 multiple times.

- (A) I only
- (B) II only
- (C) III only
- (D) I and II only
- (E) I, II, and III

11. Which of the following should be used to replace `// Line 1` in `seqSearchRecHelper` so that `seqSearchRec` will work as intended?

- (A) `if (last <= 0)`
 `return -1;`
- (B) `if (last < 0)`
 `return -1;`
- (C) `if (last < data.length)`
 `return -1;`
- (D) `while (last < data.length)`
- (E) `while (last >= 0)`

GO ON TO THE NEXT PAGE.

12. Consider the following method.

```
public String mystery(String input)
{
    String output = "";

    for (int k = 1; k < input.length(); k = k + 2)
    {
        output += input.substring(k, k + 1);
    }

    return output;
}
```

What is returned as a result of the call `mystery("computer")` ?

- (A) "computer"
- (B) "cmue"
- (C) "optr"
- (D) "ompute"
- (E) Nothing is returned because an `IndexOutOfBoundsException` is thrown.

13. Consider the following code segment.

```
int[] arr = {7, 2, 5, 3, 0, 10};
for (int k = 0; k < arr.length - 1; k++)
{
    if (arr[k] > arr[k + 1])
        System.out.print(k + "  " + arr[k] + "  ");
}
```

What will be printed as a result of executing the code segment?

- (A) 0 2 2 3 3 0
- (B) 0 7 2 5 3 3
- (C) 0 7 2 5 5 10
- (D) 1 7 3 5 4 3
- (E) 7 2 5 3 3 0

GO ON TO THE NEXT PAGE.

14. Consider the following interface and class declarations.

```
public interface Vehicle
{
    /** @return the mileage traveled by this Vehicle
     */
    double getMileage();
}

public class Fleet
{
    private ArrayList<Vehicle> myVehicles;

    /** @return the mileage traveled by all vehicles in this Fleet
     */
    public double getTotalMileage()
    {
        double sum = 0.0;

        for (Vehicle v : myVehicles)
        {
            sum += /* expression */ ;
        }

        return sum;
    }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

Which of the following can be used to replace `/* expression */` so that `getTotalMileage` returns the total of the miles traveled for all vehicles in the fleet?

- (A) `getMileage(v)`
- (B) `myVehicles[v].getMileage()`
- (C) `Vehicle.get(v).getMileage()`
- (D) `myVehicles.get(v).getMileage()`
- (E) `v.getMileage()`

GO ON TO THE NEXT PAGE.

15. Consider the following method, `isSorted`, which is intended to return `true` if an array of integers is sorted in nondecreasing order and to return `false` otherwise.

```
/** @param data an array of integers
 *  @return true if the values in the array appear in sorted (nondecreasing) order
 */
public static boolean isSorted(int[] data)
{
    /* missing code */
}
```

Which of the following can be used to replace `/* missing code */` so that `isSorted` will work as intended?

- I.

```
for (int k = 1; k < data.length; k++)
{
    if (data[k - 1] > data[k])
        return false;
}
return true;
```
- II.

```
for (int k = 0; k < data.length; k++)
{
    if (data[k] > data[k + 1])
        return false;
}
return true;
```
- III.

```
for (int k = 0; k < data.length - 1; k++)
{
    if (data[k] > data[k + 1])
        return false;
    else
        return true;
}
return true;
```

- (A) I only
- (B) II only
- (C) III only
- (D) I and II only
- (E) I and III only

16. Consider the following incomplete method that is intended to return an array that contains the contents of its first array parameter followed by the contents of its second array parameter.

```
public static int[] append(int[] a1, int[] a2)
{
    int[] result = new int[a1.length + a2.length];

    for (int j = 0; j < a1.length; j++)
        result[j] = a1[j];

    for (int k = 0; k < a2.length; k++)
        result[ /* index */ ] = a2[k];

    return result;
}
```

Which of the following expressions can be used to replace `/* index */` so that `append` will work as intended?

- (A) `j`
- (B) `k`
- (C) `k + a1.length - 1`
- (D) `k + a1.length`
- (E) `k + a1.length + 1`

17. Consider the following code segment.

```
int[] arr = {1, 2, 3, 4, 5, 6, 7};  
  
for (int k = 3; k < arr.length - 1; k++)  
    arr[k] = arr[k + 1];
```

Which of the following represents the contents of `arr` as a result of executing the code segment?

- (A) {1, 2, 3, 4, 5, 6, 7}
- (B) {1, 2, 3, 5, 6, 7}
- (C) {1, 2, 3, 5, 6, 7, 7}
- (D) {1, 2, 3, 5, 6, 7, 8}
- (E) {2, 3, 4, 5, 6, 7, 7}

18. Assume that `myList` is an `ArrayList` that has been correctly constructed and populated with objects. Which of the following expressions produces a valid random index for `myList` ?

- (A) `(int)( Math.random() * myList.size() ) - 1`
- (B) `(int)( Math.random() * myList.size() )`
- (C) `(int)( Math.random() * myList.size() ) + 1`
- (D) `(int)( Math.random() * (myList.size() + 1) )`
- (E) `Math.random(myList.size())`

19. Assume that `a` and `b` have been defined and initialized as `int` values. The expression

`!(!(a != b) && (b > 7))`

is equivalent to which of the following?

- (A) `(a != b) || (b < 7)`
- (B) `(a != b) || (b <= 7)`
- (C) `(a == b) || (b <= 7)`
- (D) `(a != b) && (b <= 7)`
- (E) `(a == b) && (b > 7)`

GO ON TO THE NEXT PAGE.

20. Consider the following method.

```
public static void arrayMethod(int nums[])
{
    int j = 0;
    int k = nums.length - 1;

    while (j < k)
    {
        int x = nums[j];
        nums[j] = nums[k];
        nums[k] = x;
        j++;
        k--;
    }
}
```

Which of the following describes what the method `arrayMethod()` does to the array `nums`?

- (A) The array `nums` is unchanged.
- (B) The first value in `nums` is copied to every location in the array.
- (C) The last value in `nums` is copied to every location in the array.
- (D) The method generates an `ArrayIndexOutOfBoundsException`.
- (E) The contents of the array `nums` are reversed.

Questions 21-25 refer to the code from the GridWorld case study. A copy of the code is provided in the Appendix.

21. Consider the design of a `Grasshopper` class that extends `Bug`. When asked to move, a `Grasshopper` moves to a randomly chosen empty adjacent location that is within the grid. If there is no empty adjacent location that is within the grid, the `Grasshopper` does not move, but turns 45 degrees to the right without changing its location.

Which method(s) of the `Bug` class should the `Grasshopper` class override so that a `Grasshopper` can behave as described above?

- I. `act()`
 - II. `move()`
 - III. `canMove()`
- (A) I only
(B) II only
(C) I and II only
(D) II and III only
(E) I, II, and III

GO ON TO THE NEXT PAGE.

22. Assume that `gus` has been defined and initialized as a `Bug` object in a class that contains the following code segment.

```
int numTurnsMade = 0;
for (int k = 1; k <= 100; k++)
{
    int dir = gus.getDirection();
    int dirTurn = dir + Location.HALF_RIGHT;
    gus.act();
    if ( /* expression */ )
        numTurnsMade++;
}
```

Which of the following could be used to replace `/* expression */` so that the variable `numTurnsMade` accurately stores the number of times that `gus` turns 45 degrees to the right?

- (A) `dir == dirTurn`
- (B) `dir == gus.getDirection()`
- (C) `dirTurn == Location.HALF_RIGHT`
- (D) `dirTurn == gus.getDirection()`
- (E) `Location.HALF_RIGHT == gus.getDirection()`

23. Consider the following method that is intended to return an `ArrayList` of all the locations in `grd` that contain actors facing in direction `dir`.

```
public ArrayList<Location> findLocsFacingDir(int dir, Grid<Actor> grd)
{
    ArrayList<Location> desiredLocs = new ArrayList<Location>();

    for (Location loc : grd.getOccupiedLocations())
    {
        if ( /* expression */ == dir )
            desiredLocs.add(loc);
    }
    return desiredLocs;
}
```

Which of the following can be used to replace `/* expression */` so that `findLocsFacingDir` will work as intended?

- (A) `loc.getDirection()`
- (B) `getDirection(loc)`
- (C) `((Actor) loc).getDirection()`
- (D) `grd(loc).getDirection()`
- (E) `grd.get(loc).getDirection()`

24. A `ColorChangingCritter` behaves like a `ChameleonCritter` but does not turn when it moves. A partial declaration for the `ColorChangingCritter` class is as follows.

```
public class ColorChangingCritter extends ChameleonCritter
{
    public void makeMove(Location loc)
    { /* missing code */ }
}
```

Which of the following replacements for `/* missing code */` will correctly implement the desired behavior?

- I. `moveTo(loc);`
- II. `super.super.makeMove(loc);`
- III. `int dir = getDirection();`
`super.makeMove(loc);`
`setDirection(dir);`

- (A) I only
- (B) III only
- (C) I and II only
- (D) I and III only
- (E) I, II, and III

25. A `MunchingCritter` acts by selecting one adjacent actor of any type, eating it (removing it from the grid), and moving to occupy its location. If there is no adjacent actor, the `MunchingCritter` moves like a normal critter. Consider the following three implementations of `MunchingCritter`.

Implementation I

```
public class MunchingCritter extends Critter
{
    private Location eatLoc; // Remember location of critter that was eaten

    public void processActors(ArrayList<Actor> actors)
    {
        if (actors.size() == 0)
            eatLoc = null;
        else
        {
            Actor selected = actors.get(0);
            eatLoc = selected.getLocation();
            selected.removeSelfFromGrid();
        }
    }

    public Location selectMoveLocation(ArrayList<Location> locs)
    {
        if (eatLoc == null)
            return super.selectMoveLocation(locs);
        else
            return eatLoc;
    }
}
```

Implementation II

```
public class MunchingCritter extends Critter
{
    private Location eatLoc; // Remember location of critter that was eaten

    public void processActors(ArrayList<Actor> actors)
    {
        if (actors.size() == 0)
            eatLoc = null;
        else
        {
            Actor selected = actors.get(0);
            eatLoc = selected.getLocation();
            selected.removeSelfFromGrid();
        }
    }

    public void makeMove(Location loc)
    {
        if (eatLoc == null)
            moveTo(loc);
        else
            moveTo(eatLoc);
    }
}
```

GO ON TO THE NEXT PAGE.

Implementation III

```
public class MunchingCriticter extends Critter
{
    private boolean hasEaten;    // Remember if this critter ate something during this step

    public void processActors(ArrayList<Actor> actors)
    {
        if (actors.size() == 0)
            hasEaten = false;
        else
        {
            Actor selected = actors.get(0);
            Location moveLoc = selected.getLocation();
            selected.removeSelfFromGrid();
            moveTo(moveLoc);
            hasEaten = true;
        }
    }

    public void makeMove(Location loc)
    {
        if (!hasEaten)
            moveTo(loc);
    }
}
```

Which of the implementations would be considered to be well designed, in that they satisfy the postconditions in Critter.java ?

- (A) I only
- (B) II only
- (C) III only
- (D) I and II only
- (E) I, II, and III

26. Assume that the array `arr` has been defined and initialized as follows.

```
int[] arr = /* initial values for the array */ ;
```

Which of the following will correctly print all of the odd integers contained in `arr` but none of the even integers contained in `arr` ?

- (A)

```
for (int x : arr)
    if (x % 2 == 1)
        System.out.println(x);
```
- (B)

```
for (int k = 1; k < arr.length; k++)
    if (arr[k] % 2 == 1)
        System.out.println(arr[k]);
```
- (C)

```
for (int x : arr)
    if (x % 2 == 1)
        System.out.println(arr[x]);
```
- (D)

```
for (int k = 0; k < arr.length; k++)
    if (arr[k] % 2 == 1)
        System.out.println(k);
```
- (E)

```
for (int x : arr)
    if (arr[x] % 2 == 1)
        System.out.println(arr[x]);
```

GO ON TO THE NEXT PAGE.

Questions 27-28 refer to the following method.

```
public static int mystery(int n)
{
    int x = 1;
    int y = 1;

    // Point A

    while (n > 2)
    {
        x = x + y;

        // Point B

        y = x - y;
        n--;
    }

    // Point C

    return x;
}
```

27. What value is returned as a result of the call `mystery(6)` ?

- (A) 1
- (B) 5
- (C) 6
- (D) 8
- (E) 13

28. Which of the following is true of method `mystery` ?

- (A) `x` will sometimes be 1 at `// Point B`.
- (B) `x` will never be 1 at `// Point C`.
- (C) `n` will never be greater than 2 at `// Point A`.
- (D) `n` will sometimes be greater than 2 at `// Point C`.
- (E) `n` will always be greater than 2 at `// Point B`.

GO ON TO THE NEXT PAGE.

29. Consider the following code segment.

```
for (int k = 1; k <= 100; k++)  
    if ((k % 4) == 0)  
        System.out.println(k);
```

Which of the following code segments will produce the same output as the code segment above?

- (A)

```
for (int k = 1; k <= 25; k++)  
    System.out.println(k);
```
- (B)

```
for (int k = 1; k <= 100; k = k + 4)  
    System.out.println(k);
```
- (C)

```
for (int k = 1; k <= 100; k++)  
    System.out.println(k % 4);
```
- (D)

```
for (int k = 4; k <= 25; k = 4 * k)  
    System.out.println(k);
```
- (E)

```
for (int k = 4; k <= 100; k = k + 4)  
    System.out.println(k);
```

30. Consider the following method.

```
public static String scramble(String word, int howFar)
{
    return word.substring(howFar + 1, word.length()) +
           word.substring(0, howFar);
}
```

What value is returned as a result of the call `scramble("compiler", 3)`?

- (A) "compiler"
- (B) "pilercom"
- (C) "ilercom"
- (D) "ilercomp"
- (E) No value is returned because an `IndexOutOfBoundsException` will be thrown.

GO ON TO THE NEXT PAGE.

31. Consider the following method.

```
public void mystery(int[] data)
{
    for (int k = 0; k < data.length - 1; k++)
        data[k + 1] = data[k] + data[k + 1];
}
```

The following code segment appears in another method in the same class.

```
int[] values = {5, 2, 1, 3, 8};
mystery(values);
for (int v : values)
    System.out.print(v + " ");
System.out.println();
```

What is printed as a result of executing the code segment?

- (A) 5 2 1 3 8
- (B) 5 7 3 4 11
- (C) 5 7 8 11 19
- (D) 7 3 4 11 8
- (E) Nothing is printed because an `ArrayIndexOutOfBoundsException` is thrown during the execution of method `mystery`.

32. Consider the following method.

```
public int compute(int n, int k)
{
    int answer = 1;

    for (int i = 1; i <= k; i++)
        answer *= n;

    return answer;
}
```

Which of the following represents the value returned as a result of the call `compute(n, k)` ?

- (A) $n*k$
- (B) $n!$
- (C) n^k
- (D) 2^k
- (E) k^n

GO ON TO THE NEXT PAGE.

33. Consider the following code segment.

```
int sum = 0;
int k = 1;
while (sum < 12 || k < 4)
    sum += k;

System.out.println(sum);
```

What is printed as a result of executing the code segment?

- (A) 6
- (B) 10
- (C) 12
- (D) 15
- (E) Nothing is printed due to an infinite loop.

34. Consider the following class declarations.

```
public class Point
{
    private double x;    // x-coordinate
    private double y;    // y-coordinate

    public Point()
    {
        x = 0;
        y = 0;
    }

    public Point(double a, double b)
    {
        x = a;
        y = b;
    }

    // There may be instance variables, constructors, and methods that are not shown.
}

public class Circle
{
    private Point center;
    private double radius;

    /** Constructs a circle where (a, b) is the center and r is the radius.
     */
    public Circle(double a, double b, double r)
    {
        /* missing code */
    }
}
```

Which of the following replacements for */* missing code */* will correctly implement the `Circle` constructor?

- I. `center = new Point();`
`radius = r;`
- II. `center = new Point(a, b);`
`radius = r;`
- III. `center = new Point();`
`center.x = a;`
`center.y = b;`
`radius = r;`

- (A) I only
- (B) II only
- (C) III only
- (D) II and III only
- (E) I, II, and III

GO ON TO THE NEXT PAGE.

35. Consider the following code segment.

```
int num = 2574;
int result = 0;

while (num > 0)
{
    result = result * 10 + num % 10;
    num /= 10;
}
System.out.println(result);
```

What is printed as a result of executing the code segment?

- (A) 2
- (B) 4
- (C) 18
- (D) 2574
- (E) 4752

36. Consider the following method.

```
public void test(int x)
{
    int y;

    if (x % 2 == 0)
        y = 3;
    else if (x > 9)
        y = 5;
    else
        y = 1;

    System.out.println("y = " + y);
}
```

Which of the following test data sets would test each possible output for the method?

- (A) 8, 9, 12
- (B) 7, 9, 11
- (C) 8, 9, 11
- (D) 8, 11, 13
- (E) 7, 9, 10

37. Consider the following code segment.

```
int x = 1;
while ( /* missing code */ )
{
    System.out.print(x + " ");
    x = x + 2;
}
```

Consider the following possible replacements for */* missing code */*.

- I. $x < 6$
- II. $x \neq 6$
- III. $x < 7$

Which of the proposed replacements for */* missing code */* will cause the code segment to print only the values 1 3 5?

- (A) I only
- (B) II only
- (C) I and II only
- (D) I and III only
- (E) I, II, and III

38. Assume that `x` and `y` have been declared and initialized with `int` values. Consider the following Java expression.

`(y > 10000) || (x > 1000 && x < 1500)`

Which of the following is equivalent to the expression given above?

- (A) `(y > 10000 || x > 1000) && (y > 10000 || x < 1500)`
- (B) `(y > 10000 || x > 1000) || (y > 10000 || x < 1500)`
- (C) `(y > 10000) && (x > 1000 || x < 1500)`
- (D) `(y > 10000 && x > 1000) || (y > 10000 && x < 1500)`
- (E) `(y > 10000 && x > 1000) && (y > 10000 && x < 1500)`

GO ON TO THE NEXT PAGE.

39. Consider the following recursive method.

```
public int recur(int n)
{
    if (n <= 10)
        return n * 2;
    else
        return recur(recur(n / 3));
}
```

What value is returned as a result of the call `recur(27)` ?

- (A) 8
- (B) 9
- (C) 12
- (D) 16
- (E) 18

GO ON TO THE NEXT PAGE.

40. Consider the following recursive method.

```
public static void whatsItDo(String str)
{
    int len = str.length();
    if (len > 1)
    {
        String temp = str.substring(0, len - 1);
        whatsItDo(temp);
        System.out.println(temp);
    }
}
```

What is printed as a result of the call `whatsItDo("WATCH")` ?

- (A) WATC
WAT
WA
W
- (B) WATCH
WATC
WAT
WA
- (C) W
WA
WAT
WATC
- (D) W
WA
WAT
WATC
WATCH
- (E) WATCH
WATC
WAT
WA
W
WA
WAT
WATC
WATCH

END OF SECTION I

**IF YOU FINISH BEFORE TIME IS CALLED,
YOU MAY CHECK YOUR WORK ON THIS SECTION.**

DO NOT GO ON TO SECTION II UNTIL YOU ARE TOLD TO DO SO.

COMPUTER SCIENCE A
SECTION II

Time—1 hour and 45 minutes

Number of questions—4

Percent of total grade—50

Directions: SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Notes:

- Assume that the classes listed in the Quick Reference found in the Appendix have been imported where appropriate.
 - Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
 - In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods may not receive full credit.
1. Consider the following partial declaration for a `WordScrambler` class. The constructor for the `WordScrambler` class takes an even-length array of `String` objects and initializes the instance variable `scrambledWords`.

```
public class WordScrambler
{
    private String[] scrambledWords;

    /** @param wordArr an array of String objects
     *      Precondition: wordArr.length is even
     */
    public WordScrambler(String[] wordArr)
    {
        scrambledWords = mixedWords(wordArr);
    }

    /** @param word1 a String of characters
     *      @param word2 a String of characters
     *      @return a String that contains the first half of word1 and the second half of word2
     */
    private String recombine(String word1, String word2)
    { /* to be implemented in part (a) */ }

    /** @param words an array of String objects
     *      Precondition: words.length is even
     *      @return an array of String objects created by recombining pairs of strings in array words
     *      Postcondition: the length of the returned array is words.length
     */
    private String[] mixedWords(String[] words)
    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

GO ON TO THE NEXT PAGE.

- (a) Write the `WordScrambler` method `recombine`. This method returns a `String` created from its two `String` parameters as follows.
- take the first half of `word1`
 - take the second half of `word2`
 - concatenate the two halves and return the new string.

For example, the following table shows some results of calling `recombine`. Note that if a word has an odd number of letters, the second half of the word contains the extra letter.

word1	word2	recombine(word1, word2)
"apple"	"pear"	"apar"
"pear"	"apple"	"peple"

Complete method `recombine` below.

```
/** @param word1 a String of characters
 *  @param word2 a String of characters
 *  @return a String that contains the first half of word1 and the second half of word2
 */
private String recombine(String word1, String word2)
```

- (b) Write the `WordScrambler` method `mixedWords`. This method creates and returns a new array of `String` objects as follows.

It takes the first pair of strings in `words` and combines them to produce a pair of strings to be included in the array returned by the method. If this pair of strings consists of `w1` and `w2`, the method should include the result of calling `recombine` with `w1` and `w2` as arguments and should also include the result of calling `recombine` with `w2` and `w1` as arguments. The next two strings, if they exist, would form the next pair to be processed by this method. The method should continue until all the strings in `words` have been processed in this way and the new array has been filled. For example, if the array `words` contains the following elements:

```
{"apple", "pear", "this", "cat"}
```

then the call `mixedWords(words)` should return the following array.

```
{"apar", "peple", "that", "cis"}
```

In writing `mixedWords`, you may call `recombine`. Assume that `recombine` works as specified, regardless of what you wrote in part (a).

Complete method `mixedWords` below.

```
/** @param words an array of String objects
 *  Precondition: words.length is even
 *  @return an array of String objects created by recombining pairs of strings in array words
 *  Postcondition: the length of the returned array is words.length
 */
private String[] mixedWords(String[] words)
```

GO ON TO THE NEXT PAGE.

2. An array of positive integer values has the *mountain* property if the elements are ordered such that successive values increase until a maximum value (the peak of the mountain) is reached and then the successive values decrease. The `Mountain` class declaration shown below contains methods that can be used to determine if an array has the mountain property. You will implement two methods in the `Mountain` class.

```
public class Mountain
{
    /** @param array an array of positive integer values
     *   @param stop the last index to check
     *   Precondition:  $0 \leq \text{stop} < \text{array.length}$ 
     *   @return true if for each  $j$  such that  $0 \leq j < \text{stop}$ ,  $\text{array}[j] < \text{array}[j + 1]$  ;
     *   false otherwise
     */
    public static boolean isIncreasing(int[] array, int stop)
    { /* implementation not shown */ }

    /** @param array an array of positive integer values
     *   @param start the first index to check
     *   Precondition:  $0 \leq \text{start} < \text{array.length} - 1$ 
     *   @return true if for each  $j$  such that  $\text{start} \leq j < \text{array.length} - 1$ ,
     *            $\text{array}[j] > \text{array}[j + 1]$ ;
     *   false otherwise
     */
    public static boolean isDecreasing(int[] array, int start)
    { /* implementation not shown */ }

    /** @param array an array of positive integer values
     *   Precondition:  $\text{array.length} > 0$ 
     *   @return the index of the first peak (local maximum) in the array, if it exists;
     *           -1 otherwise
     */
    public static int getPeakIndex(int[] array)
    { /* to be implemented in part (a) */ }

    /** @param array an array of positive integer values
     *   Precondition:  $\text{array.length} > 0$ 
     *   @return true if array contains values ordered as a mountain;
     *   false otherwise
     */
    public static boolean isMountain(int[] array)
    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

- (a) Write the Mountain method `getPeakIndex`. Method `getPeakIndex` returns the index of the first peak found in the parameter `array`, if one exists. A peak is defined as an element whose value is greater than the value of the element immediately before it and is also greater than the value of the element immediately after it. Method `getPeakIndex` starts at the beginning of the array and returns the index of the first peak that is found or -1 if no peak is found.

For example, the following table illustrates the results of several calls to `getPeakIndex`.

<code>arr</code>	<code>getPeakIndex(arr)</code>
{11, 22, 33, 22, 11}	2
{11, 22, 11, 22, 11}	1
{11, 22, 33, 55, 77}	-1
{99, 33, 55, 77, 120}	-1
{99, 33, 55, 77, 55}	3
{33, 22, 11}	-1

Complete method `getPeakIndex` below.

```
/** @param array an array of positive integer values
 *   Precondition: array.length > 0
 *   @return the index of the first peak (local maximum) in the array, if it exists;
 *           -1 otherwise
 */
public static int getPeakIndex(int[] array)
```

- (b) Write the `Mountain` method `isMountain`. Method `isMountain` returns `true` if the values in the parameter `array` are ordered as a mountain; otherwise, it returns `false`. The values in `array` are ordered as a mountain if all three of the following conditions hold.
- There must be a peak.
 - The array elements with an index smaller than the peak's index must appear in increasing order.
 - The array elements with an index larger than the peak's index must appear in decreasing order.

For example, the following table illustrates the results of several calls to `isMountain`.

<code>arr</code>	<code>isMountain(arr)</code>
<code>{1, 2, 3, 2, 1}</code>	<code>true</code>
<code>{1, 2, 1, 2, 1}</code>	<code>false</code>
<code>{1, 2, 3, 1, 5}</code>	<code>false</code>
<code>{1, 4, 2, 1, 0}</code>	<code>true</code>
<code>{9, 3, 5, 7, 5}</code>	<code>false</code>
<code>{3, 2, 1}</code>	<code>false</code>

In writing `isMountain`, assume that `getPeakIndex` works as specified, regardless of what you wrote in part (a).

Complete method `isMountain` below.

```
/** @param array an array of positive integer values
 *   Precondition: array.length > 0
 *   @return true if array contains values ordered as a mountain;
 *           false otherwise
 */
public static boolean isMountain(int[] array)
```

3. This question involves reasoning about the code from the GridWorld case study. A copy of the code is provided as part of this exam.

A `Grub` is a `Critter` that burrows from one location to another. A `Grub` knows how far away it can burrow and randomly chooses the direction in which to burrow. It burrows down from its current location and burrows up at some target location. If the target location is empty or contains a `Flower`, the `Grub` moves to this location. If the target location contains any other type of object, the `Grub` gets stuck underground and dies.

You will implement three of the methods in the following `Grub` class.

```
public class Grub extends Critter
{
    private int maxDistance;

    public Grub(int distance)
    { maxDistance = distance; }

    /** @return one of the eight direction constants from the Location class
     */
    public int getRandomDirection()
    { /* to be implemented in part (a) */ }

    /** Gets a list of possible locations for the next move. These locations must be valid
     * in the grid of this Grub. Implemented to return all locations in a random direction
     * up to and including the maximum distance that this Grub can burrow.
     * Postcondition: The state of all actors is unchanged.
     * @return a list of all locations within the maximum distance in a randomly selected direction
     */
    public ArrayList<Location> getMoveLocations()
    { /* to be implemented in part (b) */ }

    /** Selects the location for the next move.
     * Postcondition: (1) The returned location is an element of locs, this critter's current location,
     * or null. (2) The state of all actors is unchanged.
     * @param locs the possible locations for the next move
     * @return the location that was selected for the next move, or null to indicate
     * that this Grub should be removed from the grid.
     */
    public Location selectMoveLocation(ArrayList<Location> locs)
    { /* to be implemented in part (c) */ }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

GO ON TO THE NEXT PAGE.

- (a) Write the `Grub` method `getRandomDirection` that will randomly return one of the eight direction constants from the `Location` class.

Complete method `getRandomDirection` below.

```
/** @return one of the eight direction constants from the Location class
 */
public int getRandomDirection()
```

- (b) Override the `getMoveLocations` method for the `Grub` class. A `Grub` finds its potential move locations in the following manner. It randomly generates a direction and then adds to an `ArrayList` each grid location in that direction up to and including locations that are at a distance of `maxDistance` steps away from the `Grub`'s current location. Locations that are outside the grid boundaries should not be included. The method returns this `ArrayList`.

In writing `getMoveLocations`, assume that `getRandomDirection` works as specified, regardless of what you wrote in part (a).

Complete method `getMoveLocations` below.

```
/** Gets a list of possible locations for the next move. These locations must be valid
 * in the grid of this Grub. Implemented to return all locations in a random direction
 * up to and including the maximum distance that this Grub can burrow.
 * Postcondition: The state of all actors is unchanged.
 * @return a list of all locations within the maximum distance in a randomly selected direction
 */
public ArrayList<Location> getMoveLocations()
```

- (c) Override the `selectMoveLocation` method for the `Grub` class. This method chooses a random target location from `locs` and then determines the return value according to the following criteria. If the target location is empty or contains a `Flower`, the return value is the target location. Otherwise, if the target location contains any other type of object, the return value is `null`, indicating that the `Grub` dies. If `locs` is empty, the return value is the current location.

In writing `selectMoveLocation`, assume that `getRandomDirection` and `getMoveLocations` work as specified, regardless of what you wrote in part (a) and part (b). Recall that the expression `(someObj instanceof Flower)` evaluates to `true` if `someObj` is a `Flower`.

Complete method `selectMoveLocation` below.

```
/** Selects the location for the next move.
 * Postcondition: (1) The returned location is an element of locs, this critter's current location,
 * or null. (2) The state of all actors is unchanged.
 * @param locs the possible locations for the next move
 * @return the location that was selected for the next move, or null to indicate
 *         that this Grub should be removed from the grid.
 */
public Location selectMoveLocation(ArrayList<Location> locs)
```

4. A school district would like to get some statistics on its students' standardized test scores. Scores will be represented as objects of the following `ScoreInfo` class. Each `ScoreInfo` object contains a score value and the number of students who earned that score.

```
public class ScoreInfo
{
    private int score;
    private int numStudents;

    public ScoreInfo(int aScore)
    {
        score = aScore;
        numStudents = 1;
    }

    /** adds 1 to the number of students who earned this score
     */
    public void increment()
    { numStudents++; }

    /** @return this score
     */
    public int getScore()
    { return score; }

    /** @return the number of students who earned this score
     */
    public int getFrequency()
    { return numStudents; }
}
```

The following `Stats` class creates and maintains a database of student score information. The scores are stored in sorted order in the database.

```
public class Stats
{
    private ArrayList<ScoreInfo> scoreList;
    // listed in increasing score order; no two ScoreInfo objects contain the same score

    /** Records a score in the database, keeping the database in increasing score order. If no other
     * ScoreInfo object represents score, a new ScoreInfo object representing score
     * is added to the database; otherwise, the frequency in the ScoreInfo object representing
     * score is incremented.
     * @param score a score to be recorded in the list
     * @return true if a new ScoreInfo object representing score was added to the list;
     *         false otherwise
     */
    public boolean record(int score)
    { /* to be implemented in part (a) */ }

    /** Records all scores in stuScores in the database, keeping the database in increasing score order
     * @param stuScores an array of student test scores
     */
    public void recordScores(int[] stuScores)
    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

- (a) Write the `Stats` method `record` that takes a test score and records that score in the database. If the score already exists in the database, the frequency of that score is updated. If the score does not exist in the database, a new `ScoreInfo` object is created and inserted in the appropriate position so that the database is maintained in increasing score order. The method returns `true` if a new `ScoreInfo` object was added to the database; otherwise, it returns `false`.

Complete method `record` below.

```
/** Records a score in the database, keeping the database in increasing score order. If no other
 * ScoreInfo object represents score, a new ScoreInfo object representing score
 * is added to the database; otherwise, the frequency in the ScoreInfo object representing
 * score is incremented.
 * @param score a score to be recorded in the list
 * @return true if a new ScoreInfo object representing score was added to the list;
 *         false otherwise
 */
public boolean record(int score)
```


- (b) Write the `Stats` method `recordScores` that takes an array of test scores and records them in the database. The database contains at most one `ScoreInfo` object per unique score value. Each `ScoreInfo` object contains a score and an associated frequency. The database is maintained in increasing order based on the score.

In writing `recordScores`, assume that `record` works as specified, regardless of what you wrote in part (a).

Complete method `recordScores` below.

```
/** Records all scores in stuScores in the database, keeping the database in increasing score order
 * @param stuScores an array of student test scores
 */
public void recordScores(int[] stuScores)
```

STOP

END OF EXAM

AP[®] Computer Science A
Multiple-Choice Answer Key

No.	Correct Answer
1	C
2	A
3	B
4	C
5	C
6	E
7	D
8	C
9	B
10	B
11	B
12	C
13	B
14	E
15	A
16	D
17	C
18	B
19	B
20	E
21	D
22	D
23	E
24	D
25	A
26	A
27	D
28	E
29	E
30	C

No.	Correct Answer
31	C
32	C
33	E
34	B
35	E
36	C
37	D
38	A
39	D
40	C

AP[®] Computer Science A

Free-Response Scoring Guidelines

Question 1: Word Scrambler

Part A:	<code>recombine</code>	4 points
----------------	------------------------	-----------------

- +1 extract first half of `word1`
 - +1/2 attempt (must call `substring` with 2 parameters)
 - +1/2 correct
- +1 extract second half of `word2`
 - +1/2 attempt (must call `substring` with at least 1 parameter)
 - +1/2 correct
- +1 concatenate
 - +1/2 attempt
 - +1/2 correct
- +1 return “combined” string

Part B:	<code>mixedWords</code>	5 points
----------------	-------------------------	-----------------

- +1 create a result array of correct length
- +3 loop through `words` array to create mixed pairs of strings
 - +1 attempt to loop over all pairs of strings in `words`
 - +1/2 attempt
 - +1/2 correct (loses for off-by-one pairing)
 - +1 correctly combine pairs of strings (must use `recombine`)
 - +1/2 attempt
 - +1/2 correct
 - +1 store all new string pairs in result array
 - +1/2 attempt
 - +1/2 correct
- +1 return result array

AP[®] Computer Science A
Free-Response Scoring Guidelines

Question 2: Mountain

Part A:	<code>getPeakIndex</code>	4 points
----------------	---------------------------	-----------------

- +2 `loop over array`
 - +1/2 `attempt to loop over array`
 - +1/2 `access consecutive elements for comparison`
 - +1/2 `access at least 3 values`
 - +1/2 `correct loop boundaries for solution`
- +1 `compare for peak index`
 - +1/2 `attempt to compare for peak index (must compare with at least 2 other indexes)`
 - +1/2 `correctly compare for peak index`
- + 1 `return peak index`
 - +1/2 `return correct peak index if found`
 - +1/2 `return -1 if no peak is found`

Part B:	<code>isMountain</code>	5 points
----------------	-------------------------	-----------------

- +1 `call getPeakIndex method`
 - +1/2 `attempt`
 - +1/2 `correct (has correct parameter and uses returned value correctly)`
- +1 `call isIncreasing and isDecreasing methods`
 - +1/2 `attempt (must have some attempt at appropriate parameters)`
 - +1/2 `correct (has correct parameters based on result of getPeakIndex)`
- +1 `correctly process returned value of -1 from call to getPeakIndex`
`(should not call isIncreasing and isDecreasing)`
- +1 `correctly process results of getPeakIndex, isIncreasing, and isDecreasing`
- +1 `return correct boolean value`

AP[®] Computer Science A
Free-Response Scoring Guidelines

Question 3: GrubCritic (GridWorld)

Part A:	<code>getRandomDirection</code>	1 point
----------------	---------------------------------	----------------

- +1/2 correctly generate a random direction (must be one of the eight directions in the case study)
- +1/2 return result of attempt to generate a random direction

Part B:	<code>getMoveLocations</code>	5 points
----------------	-------------------------------	-----------------

- +1/2 call `getRandomDirection` method
- +1/2 create `ArrayList` to hold locations
- +3 add locations to `ArrayList`
 - +1 loop `maxDistance` times (loop bounds)
 - +1/2 attempt
 - +1/2 correct
 - +1 get next location in chosen direction
 - +1/2 attempt
 - +1/2 correct
 - +1/2 verify that location is valid
 - +1/2 correctly add location to `ArrayList`
- +1 return `ArrayList` of locations

Part C:	<code>selectMoveLocation</code>	3 points
----------------	---------------------------------	-----------------

- +1/2 correctly get random location from `locs`
- +2 1/2 return the appropriate location
 - +1/2 return current location if `locs` is empty
 - +1/2 attempt to get actor at location in grid
 - +1/2 check chosen location for empty, flower, or other
 - +1/2 return `null` if not empty or flower
 - +1/2 return correct value in all cases

Special Usage:

- 1 changes state of `GrubCritic` in either part (b) or part (c)—violation of postconditions

AP[®] Computer Science A
Free-Response Scoring Guidelines

Question 4: Score Statistics

Part A:	<code>record</code>	7 points
----------------	---------------------	-----------------

- +1 `loop over scoreList`
 - +1/2 `attempt`
 - +1/2 `correct`

- +3 `add a ScoreInfo object to scoreList for a new unique score`
 - +1 `find correct position for adding ScoreInfo object`
 - +1/2 `attempt`
 - +1/2 `correct`

 - +1 `create a new ScoreInfo object containing score`
 - +1/2 `attempt`
 - +1/2 `correct`

 - +1 `add ScoreInfo object to scoreList`
 - +1/2 `attempt to add ScoreInfo object somewhere in scoreList`
 - +1/2 `add new ScoreInfo object at correct position in all cases`

- +2 `update frequency of existing score in scoreList`
 - +1 `find an existing ScoreInfo object containing score`
 - +1/2 `attempt`
 - +1/2 `correct`

 - +1 `update frequency in ScoreInfo object containing score`
 - +1/2 `attempt to call increment method`
 - +1/2 `correct`

- +1 `return boolean value based on whether a new ScoreInfo object was added`
 - +1/2 `attempt`
 - +1/2 `correct`

Part B:	<code>recordScores</code>	2 points
----------------	---------------------------	-----------------

- +1 `correctly loop over all scores in stuScores`
 - +1/2 `attempt`
 - +1/2 `correct`

- +1 `call record method with appropriate parameter`
 - +1/2 `attempt`
 - +1/2 `correct`

AP[®] Computer Science A
Free-Response Canonical Solutions

Question 1: Word Scrambler

PART A:

```
private String recombine(String word1, String word2)
{
    return word1.substring(0, word1.length() / 2) +
           word2.substring(word2.length() / 2);
}
```

PART B:

```
private String[] mixedWords(String[] words)
{
    String[] result = new String[words.length];

    for (int k = 0; k < result.length; k = k + 2)
    {
        result[k] = recombine(words[k], words[k + 1]);
        result[k + 1] = recombine (words[k + 1], words[k]);
    }

    return result;
}
```

AP[®] Computer Science A
Free-Response Canonical Solutions

Question 2: Mountain

PART A:

```
public static int getPeakIndex(int[] array)
{
    for (int k = 1; k < array.length - 1; k++)
    {
        if (array[k - 1] < array[k] && array[k] > array[k + 1])
            return k;
    }

    return -1;
}
```

PART B:

```
public static boolean isMountain(int[] array)
{
    int peak = getPeakIndex(array);
    return (peak != -1) && isIncreasing(array, peak) &&
        isDecreasing(array, peak);
}
```


AP[®] Computer Science A
Free-Response Canonical Solutions

Question 3: GrubCritic (GridWorld)

PART A:

```
public int getRandomDirection()
{
    int turns = (int) (Math.random() * 8);
    return Location.NORTH + turns * Location.HALF_RIGHT;
}
```

Alternate solution

```
public int getRandomDirection()
{
    return 45 * (int) (Math.random() * 8);
}
```

PART B:

```
public ArrayList<Location> getMoveLocations()
{
    Location loc = getLocation();
    ArrayList<Location> result = new ArrayList<Location>();
    int direction = getRandomDirection();

    for (int k = 0; k < maxDistance; k++)
    {
        loc = loc.getAdjacentLocation(direction);
        if (getGrid().isValid(loc))
            result.add(loc);
    }

    return result;
}
```

PART C:

```
public Location selectMoveLocation(ArrayList<Location> locs)
{
    int n = locs.size();
    if (n == 0)
        return getLocation();

    int r = (int) (Math.random() * n);
    Location loc = locs.get(r);
    Actor actorAtLoc = getGrid().get(loc);
    if (actorAtLoc == null || actorAtLoc instanceof Flower)
        return loc;
    else
        return null;
}
```

AP[®] Computer Science A
Free-Response Canonical Solutions

Question 4: Score Statistics

PART A:

```
public boolean record(int score)
{
    int k = 0;
    while (k < scoreList.size() && score > scoreList.get(k).getScore())
    {
        k++;
    }

    boolean found = k < scoreList.size() &&
                    score == scoreList.get(k).getScore();
    if (found)
        scoreList.get(k).increment();
    else
        scoreList.add(k, new ScoreInfo(score));

    return found;
}
```

Alternate solution

```
public boolean record(int score)
{
    for (int k = 0; k < scoreList.size(); k++)
    {
        if (score < scoreList.get(k).getScore())
        {
            scoreList.add(k, new ScoreInfo(score));
            return true;
        }
        else if (score == scoreList.get(k).getScore())
        {
            scoreList.get(k).increment();
            return false;
        }
    }

    scoreList.add(new ScoreInfo(score));
    return true;
}
```

PART B:

```
public void recordScores(int[] stuScores)
{
    for (int score : stuScores)
        record(score);
}
```